Internship Logging & Evaluation System

Detailed Frontend Documentation

CSC 1202: Software Development Project (2026)

Frontend	React + Vite
Purpose	Explain the ILES interface and important code clearly
Audience	Beginners, students, supervisors, and technical defence panel

ILES Frontend Detailed Documentation

Cover Information

Project name: Internship Logging & Evaluation System

Short name: ILES

Course: CSC 1202: Software Development Project

Year: 2026

Frontend technology: React with Vite

Main purpose: Help students, supervisors, academic supervisors, and administrators manage internship work in a simple digital system.

1. Simple Explanation of the System

ILES is like a school internship notebook, but it is digital. Instead of writing everything on paper, a student can open the website and record what they did each week. A supervisor can check the work, give comments, and approve it. An academic supervisor can enter marks, and the system calculates the final score automatically.

The frontend is the part of the system that users see and touch. It contains the pages, buttons, forms, tables, colors, and layout. The backend is not inside this frontend folder yet, so this version stores data in the browser using localStorage.

2. What the PDF Required

The course outline said students should build an Internship Logging & Evaluation System. The frontend was designed to match these important requirements:

- User and role management
- Internship placement tracking
- Weekly logbook records
- Supervisor review workflow
- Academic evaluation
- Weighted score computation
- Dashboards and reports
- Documentation for technical defence

3. Main Users of the System

Student Intern

The student writes weekly logs, saves drafts, submits work, and views progress.

Workplace Supervisor

The workplace supervisor reviews the weekly logs, confirms practical internship work, and gives comments.

Academic Supervisor

The academic supervisor gives academic marks and checks whether the internship meets learning objectives.

Internship Administrator

The administrator manages placements, roles, reports, and overall institutional progress.

4. Pages in the Frontend

Dashboard Page

This is the home page. It introduces ILES, shows important numbers, lists modules, displays placements, and shows the 12-week course timeline.

Weekly Logs Page

This page allows a student to create a log, save it as a draft, submit it, and move it through supervisor review states.

Evaluations Page

This page calculates final evaluation scores using the formula from the course outline.

Reports Page

This page summarizes the system using statistics, workflow counts, score distribution, and final deliverable tracking.

Settings Page

This page explains the system roles and shows workflow settings that can later connect to backend rules.

5. System Workflow

The weekly log follows this journey:

```
Draft -> Submitted -> Reviewed -> Approved
```

Draft

The student is still writing and can edit the log.

Submitted

The student has sent the log for supervisor checking.

Reviewed

The supervisor has checked the log and added feedback.

Approved

The log is accepted and should no longer be edited.

6. Score Formula

The Evaluations page uses this formula:

```
Total = (Workplace Score x 0.40) + (Academic Score x 0.30) + (Logbook Score x 0.30)
```

This means:

- Workplace supervisor mark contributes 40 percent.
- Academic supervisor mark contributes 30 percent.
- Weekly logbook mark contributes 30 percent.

Example:

```
Workplace = 80
Academic = 70
Logbook = 90

Total = (80 x 0.40) + (70 x 0.30) + (90 x 0.30)
Total = 32 + 21 + 27
Total = 80
```

7. Folder Structure

```
frontend/
  docs/
    FRONTEND_DOCUMENTATION.md
    ILES_FRONTEND_DETAILED_DOCUMENTATION.md
    ILES_FRONTEND_DETAILED_DOCUMENTATION.pdf
  src/
    components/
      DashboardCard.jsx
      Navbar.jsx
      Sidebar.jsx
    data/
      courseData.js
    pages/
```

```
      Evaluations.jsx
      Home.jsx
      Reports.jsx
      Settings.jsx
      WeeklyLogs.jsx
    App.jsx
    index.css
    main.jsx
```

8. Important Code File: main.jsx

This file starts the React app.

Key code:

```
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

Easy explanation:

- document.getElementById("root") finds the empty HTML box where React will appear.
- createRoot(...) tells React to control that box.
- <App /> is the main application component.
- <StrictMode> helps React warn developers about possible mistakes during development.

9. Important Code File: App.jsx

This file controls which page opens when a user visits a URL.

Key code:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/weeklyLogs" element={<WeeklyLogs />} />
  <Route path="/Evaluations" element={<Evaluations />} />
  <Route path="/Reports" element={<Reports />} />
  <Route path="/Settings" element={<Settings />} />
</Routes>
```

Easy explanation:

- / opens the dashboard.
- /weeklyLogs opens the weekly logbook.
- /Evaluations opens the marks and scoring page.
- /Reports opens reports.
- /Settings opens roles and settings.

Why this is important:

The user does not need to reload the whole website to move between pages. React Router changes the visible page smoothly.

10. Important Code File: courseData.js

This file stores shared information from the course outline.

Key code:

```
export const workflowStates = [
  {
    key: "Draft",
    label: "Draft",
    description: "The student can still edit the weekly activity log.",
  },
  {
    key: "Submitted",
    label: "Submitted",
    description: "The log is waiting for supervisor review.",
  },
];
```

Easy explanation:

- workflowStates is a list of possible log statuses.
- Each status has a name and explanation.
- Other pages import this list instead of rewriting the same text many times.

Why this is important:

If the workflow changes later, the developer can update it in one file, and the rest of the frontend can reuse the new information.

11. Important Code File: Navbar.jsx

The Navbar is the top bar of the website.

Key code:

```
<header className="navbar">
  <div className="navbar__brand">
    <span className="navbar__logo" aria-hidden="true">
      <GraduationCap size={24} />
    </span>
    <div>
      <strong>{courseInfo.systemShortName}</strong>
      <span>{courseInfo.code} Frontend Workspace</span>
    </div>
  </div>
</header>
```

Easy explanation:

- The top bar shows the ILES name.
- It uses courseInfo from courseData.js.

- The graduation icon helps the interface feel connected to school and learning.

12. Important Code File: Sidebar.jsx

The Sidebar is the menu on the left side.

Key code:

```
const links = [
  { to: "/", label: "Dashboard", icon: Home },
  { to: "/weeklyLogs", label: "Weekly Logs", icon: FileText },
  { to: "/Evaluations", label: "Evaluations", icon: ClipboardCheck },
  { to: "/Reports", label: "Reports", icon: BarChart3 },
  { to: "/Settings", label: "Settings", icon: Settings },
];
```

Easy explanation:

- This list stores all main navigation links.
- Each item has a URL, label, and icon.
- The component loops through the list and creates clickable menu items.

Key code:

```
<NavLink
  className={({ isActive }) =>
    isActive ? "sidebar__link sidebar__link--active" : "sidebar__link"
  }
  key={to}
  to={to}
>
```

Easy explanation:

- NavLink knows which page is currently open.
- If the link is active, it adds the active CSS class.
- This makes the current page highlighted in the sidebar.

13. Important Code File: DashboardCard.jsx

This is a reusable card used for dashboard numbers.

Key code:

```
function DashboardCard({ title, value, detail, icon: Icon, tone = "blue" }) {
  return (
    <article className={`dashboard-card dashboard-card--${tone}`}>
      <div className="dashboard-card__top">
        {Icon ? (
          <span className="dashboard-card__icon" aria-hidden="true">
            <Icon size={20} strokeWidth={2.2} />
          </span>
        ) : null}
        <span>{title}</span>
      </div>
      <strong>{value}</strong>
```

```
      {detail ? <p>{detail}</p> : null}
    </article>
  );
}
```

Easy explanation:

- The card receives information through props.
- title is the card name.
- value is the big number.
- detail is the small explanation.
- icon is optional.
- tone changes the card color style.

Why this is important:

One reusable component avoids repeating the same card code many times.

14. Important Code File: Home.jsx

This is the dashboard page.

Key code:

```
const approvedLogs = initialLogs.filter((log) => log.status === "Reviewed").length;
const averageScore = Math.round(
  initialEvaluations.reduce((sum, evaluation) => sum + evaluation.total, 0) /
    initialEvaluations.length,
);
```

Easy explanation:

- approvedLogs counts logs with a reviewed status.
- averageScore adds all evaluation totals and divides by the number of evaluations.
- Math.round removes decimals so the score looks neat.

Key code:

```
{modules.map((module) => (
  <span className="module-pill" key={module}>
    {module}
  </span>
))}
```

Easy explanation:

- modules.map(...) loops through every system module.
- For each module, it creates a small pill-shaped label.
- key={module} helps React track each item.

15. Important Code File: WeeklyLogs.jsx

This is one of the most important files because it handles logbook workflow.

Key code:

```
const emptyLog = {
  week: "",
  focusArea: "",
  hours: "",
  activity: "",
  evidence: "",
  supervisorComment: "",
  status: "Draft",
};
```

Easy explanation:

- This is the starting shape of a blank weekly log form.
- The status starts as Draft.
- When the form resets, it returns to these empty values.

Key code:

```
function getSavedLogs() {
  const savedLogs = localStorage.getItem("iles-weekly-logs");
  return savedLogs ? JSON.parse(savedLogs) : initialLogs;
}
```

Easy explanation:

- The system checks if logs are saved in the browser.
- If saved logs exist, it uses them.
- If not, it uses sample logs from courseData.js.

Key code:

```
useEffect(() => {
  localStorage.setItem("iles-weekly-logs", JSON.stringify(logs));
}, [logs]);
```

Easy explanation:

- Every time logs changes, the system saves the new list in the browser.
- JSON.stringify changes JavaScript data into text that browser storage can keep.
- [logs] means this effect only runs when logs change.

Key code:

```
function updateForm(field, value) {
  setForm((currentForm) => ({
    ...currentForm,
    [field]: value,
  }));
}
```

Easy explanation:

- This updates one field in the form.
- ...currentForm keeps the old values.
- [field]: value changes only the selected field.

Key code:

```
function handleSubmit(event) {
  event.preventDefault();

  const nextLog = {
    ...form,
    id: editId ?? Date.now(),
    status: editId ? form.status : "Draft",
    supervisorComment: form.supervisorComment || "Waiting for supervisor review.",
  };
}
```

Easy explanation:

- event.preventDefault() stops the browser from refreshing the page.
- nextLog creates the log that will be saved.
- If editing, the log keeps its old id.
- If creating a new log, Date.now() gives it a unique id.
- A new log starts as Draft.

Key code:

```
function moveStatus(logId, status) {
  setLogs((currentLogs) =>
    currentLogs.map((log) =>
      log.id === logId
        ? {
            ...log,
            status,
          }
        : log,
    ),
  );
}
```

Easy explanation:

- This changes one log from one workflow state to another.
- map goes through every log.
- If the log id matches, that log gets the new status.
- If it does not match, the log stays the same.

16. Important Code File: Evaluations.jsx

This page calculates student marks.

Key code:

```
function calculateGrade(total) {
  if (total >= 80) return "A";
  if (total >= 70) return "B";
  if (total >= 60) return "C";
  if (total >= 50) return "D";
  return "Needs Support";
}
```

Easy explanation:

- This function receives a total score.
- It returns a grade based on the total.
- For example, 82 becomes A.

Key code:

```
function calculateTotal(evaluation) {
  return Math.round(
    evaluationWeights.reduce((sum, item) => {
      const rawScore = Number(evaluation[item.key] || 0);
      return sum + (rawScore * item.weight) / 100;
    }, 0),
  );
}
```

Easy explanation:

- evaluationWeights contains the 40/30/30 score weights.
- reduce adds the weighted parts together.
- Number(...) changes typed input text into a number.
- (rawScore * item.weight) / 100 calculates the weighted contribution.
- Math.round makes the final score a whole number.

Key code:

```
const liveTotal = useMemo(() => calculateTotal(form), [form]);
const liveGrade = calculateGrade(liveTotal);
```

Easy explanation:

- The total score changes while the user types.
- The grade also updates immediately.
- useMemo helps React avoid recalculating unless the form changes.

17. Important Code File: Reports.jsx

This page summarizes the system.

Key code:

```
function readStoredData(key, fallback) {
```

```
  const savedData = localStorage.getItem(key);
  return savedData ? JSON.parse(savedData) : fallback;
}
```

Easy explanation:

- The report should use real browser-saved data if available.
- If there is no saved data, it uses sample data.

Key code:

```
const approvedLogs = logs.filter((log) => log.status === "Approved").length;
const submittedLogs = logs.filter((log) => log.status !== "Draft").length;
```

Easy explanation:

- approvedLogs counts only approved logs.
- submittedLogs counts logs that are no longer drafts.
- These numbers appear on the report cards.

Key code:

```
<div className="bar-fill" style={{ width: `${item.weight}%` }} />
```

Easy explanation:

- This creates a simple bar chart.
- If the weight is 40, the bar width becomes 40%.
- This avoids needing a big chart library.

18. Important Code File: Settings.jsx

This page explains roles and workflow controls.

Key code:

```
const defaultSettings = {
  editingLock: true,
  deadlineAlerts: true,
  supervisorComments: true,
  scoreAutoCompute: true,
};
```

Easy explanation:

- These are starting settings for the prototype.
- true means the setting is switched on.

Key code:

```
function toggleSetting(key) {
  setSettings((currentSettings) => ({
```

```
    ...currentSettings,
    [key]: !currentSettings[key],
  }));
}
```

Easy explanation:

- This turns a setting on or off.
- !currentSettings[key] means the opposite of the current value.
- If the setting is true, it becomes false.
- If it is false, it becomes true.

19. Important Code File: index.css

This file controls the appearance of the whole frontend.

Key CSS:

```
.main-layout {
  display: grid;
  grid-template-columns: 18rem minmax(0, 1fr);
  align-items: start;
}
```

Easy explanation:

- The page uses CSS Grid.
- The first column is the sidebar.
- The second column is the main page content.

Key CSS:

```
.cards-container {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 1rem;
}
```

Easy explanation:

- Dashboard cards are arranged in four columns on wide screens.
- gap adds space between cards.

Key CSS:

```
@media (max-width: 780px) {
  .main-layout {
    grid-template-columns: 1fr;
  }
}
```

Easy explanation:

- This is a responsive design rule.
- On smaller screens, the sidebar and content no longer stay side by side.
- They stack into one column so the site is easier to read on phones.

20. Why localStorage Is Used

The current repository only has a frontend. There is no Django backend yet. Because of that, the frontend uses localStorage to remember logs, evaluations, and settings.

localStorage is browser memory. It keeps data even after refreshing the page.

Current keys:

```
iles-weekly-logs
iles-evaluations
iles-settings
```

Later, when Django is ready, these should be replaced with real API calls.

21. How the Frontend Can Connect to Django Later

The frontend can later replace local browser storage with API requests:

```
GET    /api/logs/
POST   /api/logs/
PATCH  /api/logs/:id/status/
GET    /api/evaluations/
POST   /api/evaluations/
GET    /api/placements/
```

This will allow many users to share the same system data instead of saving it only in one browser.

22. Commands Used to Run the Project

Install packages:

```
npm install
```

Start development server:

```
npm run dev
```

Check code quality:

```
npm run lint
```

Build the final frontend:

```
npm run build
```

23. Testing and Verification

The frontend was tested using:

```
npm run lint
npm run build
```

Both commands passed successfully after the documentation work.

24. Technical Defence Summary

When explaining this project, say:

1. The system is based on the CSC 1202 course outline.
2. React is used to build the user interface.
3. React Router controls page navigation.
4. Shared course data is stored in courseData.js.
5. The weekly log workflow follows Draft, Submitted, Reviewed, and Approved.
6. Evaluations use automatic 40/30/30 weighted scoring.
7. Reports summarize logs, placements, workflow health, and marks.
8. localStorage is used only because the Django backend is not yet connected.
9. The frontend is ready to connect to Django REST Framework APIs later.

25. Final Beginner Summary

Think of ILES as a smart internship book:

- The Dashboard is the cover page.
- Weekly Logs are the pages students write every week.
- Evaluations are the marks section.
- Reports are the summary pages.
- Settings are the rules and users section.

The code is split into small files so each part has one clear job. This makes it easier to understand, improve, and defend during presentation.